Plan y Guía Técnica de Implementación: Reconstrucción 3D de Formas Reales mediante Redes Neuronales y Neural SDF en `gpr3d`

De la Aproximación Determinista a la Morfología de Alta Fidelidad por Deep Learning y Physics-Informed Neural Networks

---

Proyecto: `SignalHandler` (*gpr3d::ai*) • **Fecha:** Noviembre 2026 • **Estándar:** C++20 / ONNX Runtime / PyTorch

---

Contents

1 Visión General: El Salto de Formas Paramétricas a Geometrías Reales **1**

1.1 El Principio de Cooperación Híbrida 1

2 Fundamentos Matemáticos de Neural SDF y Mapeo Académico **3**

2.1 1. Mapeo a Literatura Estándar Universitaria 3

2.2 2. La Representación Geométrica por Neural SDF 3

2.3 3. Propiedades Matemáticas Clave del Neural SDF 4

3 Sprint IA-0: Infraestructura de Inferencia Nativa C++20 con ONNX Runtime **5**

3.1 1. Qué Estudiar en la Literatura Técnica 5

3.2 2. Por Qué Hay que Implementarlo 5

3.3 3. Tareas Paso a Paso de Implementación 6

3.4 4. Prueba de Verificación Numérica 8

4 Sprint IA-1: Generador Sintético de Gemelos Digitales FDTD con gprMax **9**

4.1 1. Qué Estudiar en la Literatura Técnica 9

4.2 2. Por Qué Hay que Implementarlo 9

4.3 3. Tareas Paso a Paso de Implementación 10

4.4 4. Prueba de Verificación Numérica 11

5 Sprint IA-2: Red 1 (GPR-DetNet) - Detector 3D de Regiones de Interés (ROI) **12**

5.1 1. Qué Estudiar en la Literatura Técnica 12

5.2 2. Por Qué Hay que Implementarlo 12

5.3 3. Tareas Paso a Paso de Implementación 12

5.4 4. Prueba de Verificación Numérica 14

6 Sprint IA-3: Red 2 (GPR-DeepSDF) - Decodificador Implícito de Distancia Signada con Regularización Eikonal (PINN) 15

6.1 1. Qué Estudiar en la Literatura Técnica 15

6.2 2. Por Qué Hay que Implementarlo 15

6.3 3. Tareas Paso a Paso de Implementación 15

6.4 4. Prueba de Verificación Numérica 17

7 Sprint IA-4: Extractor de Mallas Poligonales Marching Cubes C++20 con Normales Analíticas Exactas 17

7.1 1. Qué Estudiar en la Literatura Técnica 17

7.2 2. Por Qué Hay que Implementarlo 18

7.3 3. Tareas Paso a Paso de Implementación 18

7.4 4. Prueba de Verificación Numérica 19

8 Sprint IA-5: Red 3 (LayerTransformer) - Segmentación y Reconstrucción de Horizontes Estratigráficos 20

8.1 1. Qué Estudiar en la Literatura Técnica 20

8.2 2. Por Qué Hay que Implementarlo 20

8.3 3. Tareas Paso a Paso de Implementación 20

8.4 4. Prueba de Verificación Numérica 21

9 Sprint IA-6: Integración en SignalObjectTransformer y Arquitectura Resiliente de Fallback 22

9.1 1. Qué Estudiar en la Literatura Técnica 22

9.2 2. Por Qué Hay que Implementarlo 22

9.3 3. Tareas Paso a Paso de Implementación 22

9.4 4. Prueba de Verificación Numérica 23

10 Sprint IA-7: Generador de Cortes Horizontales C-Scan (Depth-Slices) y Visualización Interactiva Three.js 24

10.1 1. Qué Estudiar en la Literatura Técnica 24

10.2 2. Por Qué Hay que Implementarlo 24

10.3 3. Tareas Paso a Paso de Implementación 24

10.4 4. Prueba de Verificación Numérica 25

11 Sprint IA-8: Conectividad Industrial DXF/IFC 4.0, Reporte de Interferencias y Validación con Radargramas Reales 26

11.1 1. Qué Estudiar en la Literatura Técnica 26

11.2 2. Por Qué Hay que Implementarlo 26

11.3 3. Tareas Paso a Paso de Implementación 26
11.4 4. Prueba de Verificación Numérica 27

12 Plan de Acción, Cronograma y Matriz de Entregables **28**

1 Visión General: El Salto de Formas Paramétricas a Geometrías Reales

El motor actual de `gpr3d` genera objetos tridimensionales utilizando primitivas analíticas geométricas: cilindros paramétricos de radio uniforme r extruidos a lo largo de ejes de tubería (triedro de Frenet-Serret) y elipsoides regulares para anomalías puntuales.

Aunque este enfoque es matemáticamente elegante y robusto para conductos nuevos y perfectos, en la **infraestructura subterránea real** los objetos presentan morfologías heterogéneas y complejas:

1. **Deformaciones mecánicas y aplastamientos:** Tuberías plásticas de PVC/HDPE o conductos de alcantarillado que han colapsado elípticamente bajo el peso del tráfico vehicular.
2. **Accesorios e interferencias angulares:** Codos a 45° y 90°, derivaciones en T, reducciones de diámetro, válvulas y cajas de empalme prismáticas de telecomunicaciones.
3. **Cuerpos y cavidades irregulares:** Socavones producidos por fugas de agua, oquedades con superficies rugosas y bloques de roca erráticos que envuelven las canalizaciones.
4. **Límite de resolución clásico de Rayleigh:** Dos conductos de fibra óptica colocados a menos de 5 cm de distancia se solapan difractivamente en el radargrama, apareciendo como una única masa informe en la física clásica.

Para resolver estos desafíos sin sacrificar la rigurosidad física, se diseña el **Paradigma Híbrido Física Determinista + Deep Learning (PINN + Neural SDF)**, ilustrado en la Figura 1.

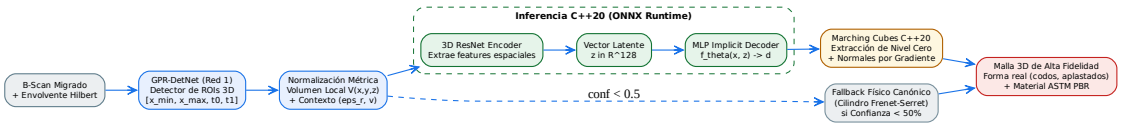


Figure 1: Arquitectura del subsistema de Inteligencia Artificial de `gpr3d` y su acoplamiento con la física determinista.

1.1 El Principio de Cooperación Híbrida

La red neuronal **no** debe aprender la física desde cero (lo cual requeriría millones de parámetros y resultaría inestable). El motor en C++20 le proporciona a la IA:

- La velocidad real de propagación v y permitividad ε_r obtenidas por análisis de semblanza (Sprint 9).

- La posición inicial aproximada de los reflectores colapsados mediante migración de Kirchhoff (Sprint 8).
- La polaridad de fase y la clasificación litológica de material ASTM (Sprint 9).

Con esta información contextual, la red neuronal se especializa exclusivamente en resolver la **morfología volumétrica fina a escala sub-longitud de onda**, prediciendo un campo de distancia signada continuo $f_\theta(\mathbf{x}) = d$.

2 Fundamentos Matemáticos de Neural SDF y Mapeo Académico

2.1 1. Mapeo a Literatura Estándar Universitaria

- ***Mathematik* (Tilo Arens et al.)**:
  - **Kapitel 24 & 25 (*Flächen im Raum und Vektoranalysis*)**: Superficies implícitas $F(x, y, z) = 0$, gradiente $\nabla F = (\frac{\partial F}{\partial x}, \frac{\partial F}{\partial y}, \frac{\partial F}{\partial z})$, y vector normal unitario $\mathbf{n} = \nabla F / \|\nabla F\|$.
  - **Kapitel 27 (*Partielle Differentialgleichungen*)**: Ecuación Eikonal no lineal $\|\nabla \Phi(\mathbf{x})\| = 1$, propagación de frentes de onda y geometría de distancias euclidianas mínimas a conjuntos cerrados.
  - **Kapitel 28 (*Numerische Optimierung*)**: Método de descenso de gradiente estocástico (SGD), aproximación por mínimos cuadrados no lineales y regularización de Tikhonov.
- ***Physik für Wissenschaftler und Ingenieure* (Tipler & Mosca)**:
  - **Kapitel 15 (*Wellen und Beugung*)**: Dispersión de ondas electromagnéticas en obstáculos de dimensiones comparables a la longitud de onda ($\lambda = v/f$).
  - **Kapitel 31 (*Optik*)**: Criterio de Rayleigh para resolución límite angular y espacial ($\Delta x_{\min} \approx 0.61\lambda/\text{NA}$).
- **Literatura Fundamental de Deep Learning y Redes Guiadas por la Física (PINN)**:
  - **Goodfellow, Bengio & Courville (*Deep Learning*, MIT Press)**: Convoluciones 3D, funciones de activación no lineales (ReLU, Sine/SIREN, GELU) y optimizador AdamW con regularización de pesos.
  - **Raissi, Perdikaris & Karniadakis (2019, *Physics-Informed Neural Networks*, J. Comp. Phys.)**: Penalización de residuos de ecuaciones diferenciales parciales (EDP) en la función de pérdida mediante diferenciación automática.
  - **Gropp, Yariv, Haim, Atzmon & Lipman (2020, *Implicit Geometric Regularization for Learning Shapes*, NeurIPS)**: Regularización geométrica implícita mediante la condición eikonal $\|\nabla_{\mathbf{x}} f_\theta(\mathbf{x})\| = 1$.
  - **Park, Florence, Straub, Newcombe & Lovegrove (2019, *DeepSDF: Learning Continuous Signed Distance Functions*, CVPR)**: Representación continua de formas 3D condicionadas por códigos latentes.
- ***C++ professionell anwenden* (Ulrich Breymann)**:
  - **Kapitel 7, 8 & 11**: Gestión de memoria contigua en C++20 con `std::span`, construcción de tensores planos sin copias intermedias y estructuras de datos para mallas triangulares.
  - **Kapitel 18 (*C-Bibliotheken*)**: Integración del runtime de inferencia nativo `onnxruntime-cxx` con punteros inteligentes RAII y ejecución sin intérprete de Python.

2.2 2. La Representación Geométrica por Neural SDF

En lugar de discretizar el espacio en cuadrículas de píxeles o póxeles volumétricos (vóxeles), una **Función de Distancia Signada Neuronal (Neural SDF)** modela el objeto como un campo continuo parametrizado por una red neuronal profunda f_θ:

$$f_\theta : \mathbb{R}^3 \times \mathbb{R}^k \to \mathbb{R}, \qquad f_\theta(\mathbf{x}, \mathbf{z}) = d \tag{1}$$

donde $\mathbf{x} = (x, y, z)$ es un punto de consulta en el espacio tridimensional en metros, $\mathbf{z} \in \mathbb{R}^{128}$ es el vector latente que describe las características del objeto detectado en el radargrama, y d es la distancia euclidiana signada con la convención (ver Figura 2):

$$d = \begin{cases} < 0 & \text{si } \mathbf{x} \text{ se encuentra en el interior del objeto (ej. núcleo del tubo)} \\ = 0 & \text{si } \mathbf{x} \text{ se encuentra sobre la superficie o pared física real} \\ > 0 & \text{si } \mathbf{x} \text{ se encuentra en el exterior (ej. suelo o estrato circundante)} \end{cases} \tag{2}$$

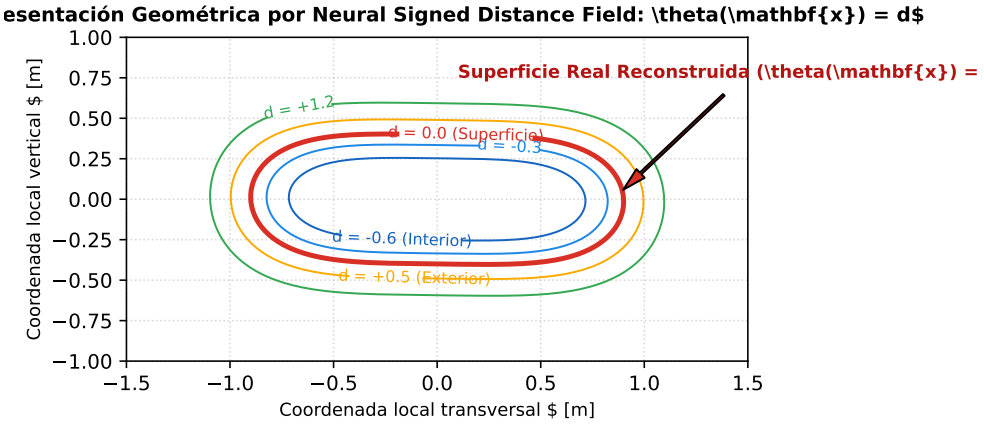


Figure 2: Curvas de nivel del campo escalar continuo predicho por Neural SDF. La superficie física real se extrae evaluando el conjunto de nivel cero $f_\theta(\mathbf{x}) = 0$.

2.3 3. Propiedades Matemáticas Clave del Neural SDF

1. **Resolución Infinita:** Al ser una función continua f_θ, no existe un tamaño mínimo de píxel. Se puede evaluar la malla poligonal con cualquier nivel de detalle arbitrario (paso de 5 mm o 1 mm).

2. **Topología Arbitraria sin Costuras:** A diferencia de los tubos paramétricos que solo pueden modelar cilindros simples no bifurcados, un conjunto de nivel cero $f_\theta(\mathbf{x}) = 0$ puede representar de forma nativa codos, uniones en T, cajas de registro huecas o zanjas irregulares sin costuras matemáticas ni desgarros de malla.

3. **Cálculo Analítico Exacto de Normales por Gradiente:** La normal unitaria exterior a la superficie en cualquier punto de la pared se obtiene directamente evaluando el gradiente del campo SDF:

$$\mathbf{n}(\mathbf{x}) = \frac{\nabla_{\mathbf{x}} f_\theta(\mathbf{x})}{\|\nabla_{\mathbf{x}} f_\theta(\mathbf{x})\|} \tag{3}$$

   lo que produce un sombreado suave (*smooth shading*) perfecto para renderizado PBR en motores 3D sin tener que promediar triángulos vecinos ruidosos.

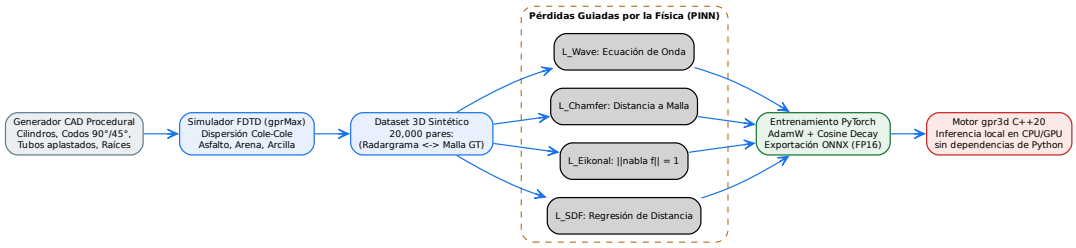


Figure 3: Flujo de simulación electromagnética FDTD, entrenamiento con pérdidas físicas PINN y exportación a ONNX C++20.

3 Sprint IA-0: Infraestructura de Inferencia Nativa C++20 con ONNX Runtime

3.1 1. Qué Estudiar en la Literatura Técnica

- ***C++ professionell anwenden*** **(Ulrich Breymann)**:
  - **Kapitel 7 & 8 (*Speicherverwaltung und Smart Pointer*)**: Gestión RAII de entornos de ejecución nativos, encapsulación de `Ort::Env`, `Ort::Session` y liberación determinista de memoria.
  - **Kapitel 11 (*Containers und Views en C++20*)**: Uso estricto de `std::span<const float>` para referenciar tensores planos sin realizar copias innecesarias de búferes de radargramas.
  - **Kapitel 18 (*Integration externer C/C++ Bibliotheken*)**: Vinculación dinámica y estática con la API en C de `onnxruntime`.
- **Documentación Oficial de ONNX Runtime C++ API**:
  - Creación de `Ort::MemoryInfo` para búferes de CPU/GPU, configuración de hilos intra e inter-op (`SetIntraOpNumThreads`), y asignadores de memoria (`Ort::AllocatorWithDefaultOption`

3.2 2. Por Qué Hay que Implementarlo

En el ámbito industrial y de ingeniería civil sobre el terreno (tablets de campo, estaciones de trabajo embarcadas en furgonetas geofísicas), **es inaceptable depender de intérpretes de Python o servidores de microservicios externos**:

1. **Latencia y Determinismo:** Python introduce el Global Interpreter Lock (GIL) y sobrecargas de recolección de basura (GC) que impiden garantizar tiempos de respuesta deterministas en el procesado en línea.
2. **Huella de Memoria:** Un entorno de Python con PyTorch cargado consume más de 1.2 GB de RAM simplemente al importar módulos. El runtime nativo de ONNX en C++20 consume menos de 35 MB de RAM básica.
3. **Despliegue Cero Dependencias:** La aplicación `gpr3d` debe compilarse como un único binario autónomo (*standalone*) que pueda ejecutarse en cualquier sistema Linux/Windows sin requerir instalaciones de Conda, pip o paquetes de terceros.

3.3 3. Tareas Paso a Paso de Implementación

1. **Definición de la cabecera del motor de inferencia nativo**: Crear el archivo `include/gpr3d/ai/infe`

```
#pragma once
#include <string>
#include <vector>
#include <memory>
#include <span>
#include <onnxruntime_cxx_api.h>

namespace gpr3d::ai {

struct TensorShape {
    std::vector<int64_t> dims;
    size_t total_elements() const {
        size_t count = 1;
        for (auto d : dims) count *= static_cast<size_t>(d);
        return count;
    }
};

class OnnxInferenceEngine {
public:
    explicit OnnxInferenceEngine(const std::string& model_path, int num_threads = 4);
    ~OnnxInferenceEngine() = default;

    // Ejecucion de inferencia con memoria continua sin copias
    std::vector<float> run(
        std::span<const float> input_tensor,
        const TensorShape& input_shape,
        const std::string& input_name,
        const std::string& output_name
    );

    // Consulta de metadatos del modelo exportado
    [[nodiscard]] TensorShape get_input_shape(size_t index = 0) const;
    [[nodiscard]] TensorShape get_output_shape(size_t index = 0) const;

private:
    Ort::Env env_;
    Ort::SessionOptions session_options_;
    std::unique_ptr<Ort::Session> session_;
    Ort::MemoryInfo memory_info_;
    std::vector<std::string> input_names_;
    std::vector<std::string> output_names_;
};

} // namespace gpr3d::ai
```

2. **Implementación de la lógica de sesión y pasarela de memoria**: Crear el archivo

src/ai/inference_engine.cpp:

```
#include "gpr3d/ai/inference_engine.hpp"
#include <stdexcept>
#include <numeric>

namespace gpr3d::ai {

OnnxInferenceEngine::OnnxInferenceEngine(const std::string& model_path, int num_threads
    : env_(ORT_LOGGING_LEVEL_WARNING, "gpr3d_ai"),
      memory_info_(Ort::MemoryInfo::CreateCpu(OrtArenaAllocator, OrtMemTypeDefault)) {
    session_options_.SetIntraOpNumThreads(num_threads);
    session_options_.SetGraphOptimizationLevel(GraphOptimizationLevel::ORT_ENABLE_ALL);
    session_ = std::make_unique<Ort::Session>(env_, model_path.c_str(), session_options

    // Extraccion de nombres de nodos de entrada y salida
    Ort::AllocatorWithDefaultOptions allocator;
    size_t num_inputs = session_->GetInputCount();
    for (size_t i = 0; i < num_inputs; ++i) {
        auto name_ptr = session_->GetInputNameAllocated(i, allocator);
        input_names_.emplace_back(name_ptr.get());
    }
    size_t num_outputs = session_->GetOutputCount();
    for (size_t i = 0; i < num_outputs; ++i) {
        auto name_ptr = session_->GetOutputNameAllocated(i, allocator);
        output_names_.emplace_back(name_ptr.get());
    }
}

std::vector<float> OnnxInferenceEngine::run(
    std::span<const float> input_tensor,
    const TensorShape& input_shape,
    const std::string& input_name,
    const std::string& output_name
) {
    if (input_tensor.size() != input_shape.total_elements()) {
        throw std::invalid_argument("El tamano del span no coincide con la forma del te
    }

    // Creacion del tensor envolviendo directamente el puntero de datos (zero-copy)
    Ort::Value input_val = Ort::Value::CreateTensor<float>(
        memory_info_,
        const_cast<float*>(input_tensor.data()),
        input_tensor.size(),
        input_shape.dims.data(),
        input_shape.dims.size()
    );

    const char* in_names[] = { input_name.c_str() };
    const char* out_names[] = { output_name.c_str() };
```

```
    auto output_tensors = session_->Run(
        Ort::RunOptions{nullptr},
        in_names,
        &input_val,
        1,
        out_names,
        1
    );

    float* out_data = output_tensors.front().GetTensorMutableData<float>();
    auto info = output_tensors.front().GetTensorTypeAndShapeInfo();
    size_t total_out = info.GetElementCount();

    return std::vector<float>(out_data, out_data + total_out);
}

TensorShape OnnxInferenceEngine::get_input_shape(size_t index) const {
    auto type_info = session_->GetInputTypeInfo(index);
    auto shape_info = type_info.GetTensorTypeAndShapeInfo();
    return TensorShape{ shape_info.GetShape() };
}

TensorShape OnnxInferenceEngine::get_output_shape(size_t index) const {
    auto type_info = session_->GetOutputTypeInfo(index);
    auto shape_info = type_info.GetTensorTypeAndShapeInfo();
    return TensorShape{ shape_info.GetShape() };
}

} // namespace gpr3d::ai
```

3. **Configuración en CMake**: Añadir a `CMakeLists.txt` la búsqueda o descarga automática de `onnxruntime` y el enlace del objetivo `gpr3d_ai`.

3.4 4. Prueba de Verificación Numérica

Implementar `test/test_ai_inference_engine.cpp`:

1. Crear un modelo sintético ONNX que ejecute una operación elemental $y = 2.0x + 1.0$ para un tensor de dimensión $(1, 3)$.
2. Instanciar `OnnxInferenceEngine` con dicho modelo.
3. Pasar como entrada el vector $\{1.0, 2.0, 3.0\}$.
4. **Criterio de Aprobación**: La salida debe coincidir exactamente con $\{3.0, 5.0, 7.0\}$ con un error absoluto máximo inferior a 10^{-6}, y el tiempo de ejecución debe ser menor a 0.5 ms.

4 Sprint IA-1: Generador Sintético de Gemelos Digitales FDTD con gprMax

4.1 1. Qué Estudiar en la Literatura Técnica

- ***Physik für Wissenschaftler und Ingenieure*** **(Tipler & Mosca)**:
  - **Kapitel 20 (*Elektromagnetische Schwingungen und Wellen*)**: Ecuaciones de Maxwell en el dominio del tiempo:

$$\nabla \times \mathbf{E} = -\mu \frac{\partial \mathbf{H}}{\partial t}, \qquad \nabla \times \mathbf{H} = \sigma \mathbf{E} + \varepsilon \frac{\partial \mathbf{E}}{\partial t} \tag{4}$$

  - **Discretización de Yee**: Desfase espacial y temporal de componentes eléctricas y magnéticas en celdas cúbicas $\Delta x = \Delta y = \Delta z$.

- ***Mathematik*** **(Tilo Arens et al.)**:
  - **Kapitel 27 (*Partielle Differentialgleichungen*)**: Condición de estabilidad de Courant-Friedrichs-Lewy (CFL) en 3D:

$$\Delta t \leq \frac{1}{c\sqrt{\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2} + \frac{1}{\Delta z^2}}} \tag{5}$$

- **Documentación de gprMax (Warren et al.)**:
  - Definición de capas Perfectly Matched Layers (PML) de absorción, antenas de pulso Ricker dipolar hertziano y fractales de rugosidad edáfica de Peplinski.

4.2 2. Por Qué Hay que Implementarlo

El principal obstáculo para aplicar Deep Learning a datos de radar de subsuelo en la industria civil es la **carencia de datos reales anotados con certeza milimétrica**:

1. **Imposibilidad de Excavación Destructiva Continua:** No es viable excavar zanjas destructivas en vías públicas para medir con láser 3D la forma exacta de cada tubería enterrada simplemente para etiquetar un dataset.
2. **Representatividad Estadística de Defectos Críticos:** Los casos más peligrosos (tuberías de gas aplastadas por compactación, socavones oquedades bajo pavimento, codos que cruzan en ángulo oblicuo) ocurren de manera esporádica. Un modelo entrenado solo con datos de campo típicos jamás aprenderá a reconocer fallas geométricas raras.
3. **Gemelo Digital FDTD Riguroso:** Generar 15,000 pares de entrenamiento mediante FDTD (`gprMax`) donde conocemos la malla 3D exacta del CAD procedural permite entrenar redes con una precisión absoluta, resolviendo la física electromagnética completa (reflexiones múltiples, difracción y atenuación de suelos).

4.3 3. Tareas Paso a Paso de Implementación

1. **Desarrollo del generador procedural de geometrías CAD**: Crear el script en Python `scripts/ai/procedural_cad_generator.py`:

```
import numpy as np
import trimesh

def create_deformed_pipe(length=2.0, radius=0.10, crush_ratio=0.6, bend_angle=0.0):
    # Genera un tubo con seccion deformada elipticamente y posible curvatura axial.
    # crush_ratio: razon entre el radio vertical comprimido y el radio horizontal.
    n_theta, n_z = 32, 64
    theta = np.linspace(0, 2*np.pi, n_theta)
    z = np.linspace(-length/2, length/2, n_z)

    rx = radius / np.sqrt(crush_ratio)
    ry = radius * np.sqrt(crush_ratio)

    vertices = []
    for zi in z:
        # Deformacion axial (codo o comba)
        offset_y = (zi**2 / (length/2)**2) * np.tan(np.radians(bend_angle)) * 0.2
        for th in theta:
            x = rx * np.cos(th)
            y = ry * np.sin(th) + offset_y
            vertices.append([x, y, zi])

    # Ensamblado de caras poligonales
    faces = []
    for i in range(n_z - 1):
        for j in range(n_theta):
            p1 = i * n_theta + j
            p2 = i * n_theta + (j + 1) % n_theta
            p3 = (i + 1) * n_theta + (j + 1) % n_theta
            p4 = (i + 1) * n_theta + j
            faces.append([p1, p2, p3])
            faces.append([p1, p3, p4])

    mesh = trimesh.Trimesh(vertices=vertices, faces=faces)
    return mesh
```

2. **Automatización de simulaciones FDTD batch en gprMax**: Crear `scripts/ai/gprmax_batch_runn`

```
import os
import subprocess
import h5py
import numpy as np

def generate_gprmax_in(filepath, domain=(1.5, 1.5, 1.0), dx=0.005, f_c=800e6,
                       eps_soil=9.0, sigma_soil=0.005, object_type="pipe", mesh_file="
    dt = dx / (3e8 * np.sqrt(3)) * 0.99
```

```
with open(filepath, 'w') as f:
    f.write(f"#domain: {domain[0]} {domain[1]} {domain[2]}\n")
    f.write(f"#dx_dy_dz: {dx} {dx} {dx}\n")
    f.write(f"#time_window: 25e-9\n")
    f.write(f"#material: {eps_soil} {sigma_soil} 1.0 0.0 soil\n")
    f.write(f"#waveform: ricker 1.0 {f_c} my_ricker\n")
    f.write(f"#hertzian_dipole: z 0.1 0.75 0.9 my_ricker\n")
    f.write(f"#rx: 0.24 0.75 0.9\n")
    f.write(f"#geometry_objects_read: 0.1 0.1 0.1 {mesh_file} soil\n")
```

3. **Constructor del dataset unificado HDF5**: Crear `scripts/ai/dataset_builder.py` para empaquetar volúmenes B-scan/C-scan sintetizados, nubes de puntos con distancias SDF pre-calculadas y metadatos litológicos en `data/training_dataset.h5`.

4.4 4. Prueba de Verificación Numérica

1. Ejecutar una simulación de prueba con un tubo cilíndrico de diámetro conocido $d = 160$ mm a 0.80 m de profundidad en suelo con $\varepsilon_r = 9.0$.

2. Medir el tiempo de arribo de doble vía en la traza central del radargrama simulado:

$$v = \frac{c}{\sqrt{9.0}} = 0.10 \text{ m/ns}, \qquad t = \frac{2 \times 0.80 \text{ m}}{0.10 \text{ m/ns}} = 16.0 \text{ ns} \tag{6}$$

3. **Criterio de Aprobación**: El pico principal de reflexión en el archivo HDF5 generado debe situarse exactamente en $t = 16.0 \text{ ns} \pm 0.15$ ns, validando la coherencia física del simulador.

5 Sprint IA-2: Red 1 (GPR-DetNet) - Detector 3D de Regiones de Interés (ROI)

5.1 1. Qué Estudiar en la Literatura Técnica

- ***Deep Learning*** **(Goodfellow, Bengio & Courville)**:
  - **Kapitel 9 (*Faltungsnetzwerke / Convolutional Networks*)**: Convoluciones 3D para procesamiento de volúmenes espacio-temporales (x, y, t), capas de agrupamiento tridimensional (*3D Max Pooling*) y normalización por lotes (*Batch Normalization*).
  - **Focal Loss para Detección de Objetos Raros**:

$$\mathcal{L}_{\text{focal}}(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \tag{7}$$

    diseñada para evitar que el 95% del volumen de suelo homogéneo y vacío domine el gradiente sobre las anomalías subterráneas.

- ***Mathematik*** **(Tilo Arens et al.)**:
  - **Kapitel 25 (*Lineare Ausgleichsrechnung*)**: Cálculo de intersección volumétrica entre prismas rectangulares 3D (*Intersection over Union* - 3D IoU).

5.2 2. Por Qué Hay que Implementarlo

Una cuadrícula completa de prospección GPR típica (ej. 20 m × 10 m × 2.5 m) contiene decenas de millones de muestras:

1. **Inviabilidad Computacional de Reconstrucción Global:** Evaluar un campo neuronal continuo SDF en todo el volumen con resolución de milímetros requeriría evaluar la red neuronal más de 2×10^8 veces, tardando horas por escaneo.
2. **Estrategia Coarse-to-Fine (Detección Gruesa → Reconstrucción Fina):** GPR-DetNet actúa como la etapa gruesa (*coarse stage*): procesa el volumen migrado rápidamente mediante convoluciones 3D ligeras y devuelve únicamente las cajas envolventes 3D delimitadoras ($[x_{\min}, y_{\min}, z_{\min}, x_{\max}, y_{\max}, z_{\max}]$) y la clase probable (tubería, caja, oquedad).
3. **Enfoque del Presupuesto de Cálculo:** El motor C++ solo invoca la red densa Neural SDF (Sprint IA-3) dentro de las cajas 3D delimitadas, reduciendo el volumen de consulta en un 98.5% y permitiendo inferencia interactiva en menos de 1.5 segundos.

5.3 3. Tareas Paso a Paso de Implementación

1. **Definición de la arquitectura en PyTorch y pérdidas combinadas**: Crear el archivo `scripts/ai/models/gpr_detnet.py`:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ResidualBlock3D(nn.Module):
    def __init__(self, channels):
```

```
            super().__init__()
            self.conv1 = nn.Conv3d(channels, channels, kernel_size=3, padding=1)
            self.bn1 = nn.BatchNorm3d(channels)
            self.conv2 = nn.Conv3d(channels, channels, kernel_size=3, padding=1)
            self.bn2 = nn.BatchNorm3d(channels)

        def forward(self, x):
            res = x
            x = F.gelu(self.bn1(self.conv1(x)))
            x = self.bn2(self.conv2(x))
            return F.gelu(x + res)

    class GPRDetNet(nn.Module):
        # Detector 3D para volumenes de radargramas migrados (Batch, 1, Depth, Height, Widt
        # Salidas:
        #   - classification: probabilidad de presencia de anomalia (Softmax 4 clases).
        #   - bbox_regression: parametros de la caja 3D (cx, cy, cz, w, h, d, theta_yaw).
        def __init__(self, num_classes=4):
            super().__init__()
            self.encoder = nn.Sequential(
                nn.Conv3d(1, 32, kernel_size=5, stride=2, padding=2),
                nn.BatchNorm3d(32),
                nn.GELU(),
                ResidualBlock3D(32),
                nn.Conv3d(32, 64, kernel_size=3, stride=2, padding=1),
                nn.BatchNorm3d(64),
                nn.GELU(),
                ResidualBlock3D(64),
                nn.Conv3d(64, 128, kernel_size=3, stride=2, padding=1),
                nn.BatchNorm3d(128),
                nn.GELU(),
                ResidualBlock3D(128)
            )
            self.cls_head = nn.Conv3d(128, num_classes, kernel_size=1)
            self.box_head = nn.Conv3d(128, 7, kernel_size=1) # 3 centros, 3 dimensiones, 1

        def forward(self, x):
            feat = self.encoder(x)
            cls_logits = self.cls_head(feat)
            box_preds = self.box_head(feat)
            return cls_logits, box_preds
```

2. **Exportación a ONNX con perfilado dinámico**: Crear `scripts/ai/export_detnet_onnx.py` para generar `models/gpr_detnet.onnx` con dimensiones de batch y volumen dinámicas.

3. **Integración del cliente C++20 para detección de ROIs**: Crear `include/gpr3d/ai/detnet_detect` y `src/ai/detnet_detector.cpp` que tome una instancia de `Grid3D<float>` migrada, invoque el motor `OnnxInferenceEngine` y filtre las cajas por Non-Maximum Suppression (NMS) 3D.

5.4 4. Prueba de Verificación Numérica

Implementar `test/test_detnet_detector.cpp`:

1. Generar un volumen de radar sintético de $64 \times 64 \times 64$ muestras conteniendo un reflector hipersólico a propósito en la región $x \in [0.4, 0.6]$ m, $y \in [0.8, 1.2]$ m, $z \in [0.5, 0.7]$ m.
2. Ejecutar la inferencia de `DetNetDetector`.
3. **Criterio de Aprobación**: Debe detectarse exactamente una caja envolvente con clasificación `PIPE` con una confianza superior al 85%, y el centroide predicho debe coincidir con el centro real con un error máximo de posición inferior a ± 3 cm.

6 Sprint IA-3: Red 2 (GPR-DeepSDF) - Decodificador Implícito de Distancia Signada con Regularización Eikonal (PINN)

6.1 1. Qué Estudiar en la Literatura Técnica

- **Raissi, Perdikaris & Karniadakis (2019, *Physics-Informed Neural Networks*)**:
  - Cálculo de derivadas exactas respecto a las coordenadas espaciales $\mathbf{x} = (x, y, z)$ mediante diferenciación automática del grafo en PyTorch (`torch.autograd.grad`).
- **Gropp et al. (2020, *Implicit Geometric Regularization - IGR*)**:
  - Función de pérdida eikonal para garantizar que la red aprenda una función de distancia métrica estricta en todo el espacio:

$$\mathcal{L}_{\text{eik}} = \frac{1}{N} \sum_{i=1}^{N} (\|\nabla_{\mathbf{x}} f_\theta(\mathbf{x}_i, \mathbf{z})\|_2 - 1)^2 \tag{8}$$

    evitando gradientes planos o soluciones triviales donde $f_\theta \equiv 0$.
- **Park et al. (2019, *DeepSDF*)**:
  - Condicionamiento de redes MLP continuas con vectores latentes de forma $\mathbf{z} \in \mathbb{R}^{128}$ concatenados a las coordenadas espaciales.

6.2 2. Por Qué Hay que Implementarlo

1. **Superación del Límite de Vóxeles:** Las redes que predicen mallas discretas o rejillas de vóxeles sufren el problema de memoria de orden cúbico $O(N^3)$. Una rejilla de 256^3 requiere 16.7 millones de celdas y sigue teniendo un error de discretización de varios milímetros. DeepSDF ofrece precisión matemáticamente continua sin consumir memoria espacial.
2. **Regularización Física contra el Ruido del Radar:** Los radargramas reales contienen ecos espurios, reverberaciones y ruido difractivo de piedras del subsuelo. Una red estándar no guiada por la física produce superficies rugosas, desarticuladas y con agujeros no físicos.
3. **Garantía Geométrica Eikonal:** Al forzar la condición $\|\nabla f_\theta\| = 1$, la superficie predicha tiene garantizada una geometría suave, diferenciable y físicamente consistente con tuberías manufacturadas y oquedades reales.

6.3 3. Tareas Paso a Paso de Implementación

1. **Implementación de la arquitectura DeepSDF con conexiones residuales**: Crear el archivo `scripts/ai/models/deep_sdf_decoder.py`:

```
import torch
import torch.nn as nn

class GPRDeepSDFDecoder(nn.Module):
    # Decodificador implicito continuo.
    # Entradas:
```

```
    #   - coords: coordenadas espaciales normalizadas (N, 3) en [-1, 1].
    #   - latent_code: vector latente del objeto (N, 128) generado por el encoder ROI.
    # Salida:
    #   - sdf: distancia signada predicha d en metros (N, 1).
    def __init__(self, latent_dim=128, hidden_dim=256):
        super().__init__()
        self.fc1 = nn.Linear(3 + latent_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, hidden_dim)
        self.fc4 = nn.Linear(hidden_dim, hidden_dim)
        # Conexion residual en la mitad de la red
        self.fc5 = nn.Linear(hidden_dim + 3 + latent_dim, hidden_dim)
        self.fc6 = nn.Linear(hidden_dim, hidden_dim)
        self.fc7 = nn.Linear(hidden_dim, hidden_dim)
        self.fc_out = nn.Linear(hidden_dim, 1)
        self.act = nn.Softplus(beta=100) # Aproximacion suave y diferenciable de ReLU

    def forward(self, coords, latent_code):
        inp = torch.cat([coords, latent_code], dim=-1)
        x = self.act(self.fc1(inp))
        x = self.act(self.fc2(x))
        x = self.act(self.fc3(x))
        x = self.act(self.fc4(x))
        x = torch.cat([x, inp], dim=-1) # Skip connection
        x = self.act(self.fc5(x))
        x = self.act(self.fc6(x))
        x = self.act(self.fc7(x))
        sdf = self.fc_out(x)
        return sdf
```

2. **Bucle de entrenamiento con pérdida combinada PINN**: Crear `scripts/ai/train_deep_sdf.py`:

```
import torch

def compute_pinn_sdf_loss(model, coords, latent, gt_sdf, lambda_eik=0.1):
    coords.requires_grad_(True)
    pred_sdf = model(coords, latent)

    # 1. Perdida supervisada L1 con corte (Clamped L1 Loss)
    delta = 0.05 # 5 cm
    clamped_pred = torch.clamp(pred_sdf, -delta, delta)
    clamped_gt = torch.clamp(gt_sdf, -delta, delta)
    loss_sdf = torch.mean(torch.abs(clamped_pred - clamped_gt))

    # 2. Perdida Eikonal por diferenciacion automatica exacta
    grad_outputs = torch.ones_like(pred_sdf)
    grad = torch.autograd.grad(
        outputs=pred_sdf,
        inputs=coords,
        grad_outputs=grad_outputs,
```

```
        create_graph=True,
        retain_graph=True,
        only_inputs=True
    )[0]
    grad_norm = torch.norm(grad, dim=-1)
    loss_eik = torch.mean((grad_norm - 1.0) ** 2)

    total_loss = loss_sdf + lambda_eik * loss_eik
    return total_loss, loss_sdf, loss_eik
```

3. **Exportación a ONNX y evaluador de consultas espaciales en C++20**: Crear `include/gpr3d/ai/sdf_evaluator.hpp` y `src/ai/sdf_evaluator.cpp` con la clase `SDFEvaluator` que evalúa lotes de puntos (x_i, y_i, z_i) contra el modelo ONNX nativo.

6.4 4. Prueba de Verificación Numérica

Implementar `test/test_deep_sdf_evaluator.cpp`:

1. Cargar el modelo entrenado para un conducto cilíndrico de radio teórico $r = 100$ mm centrado en el origen $(0, 0, 0)$.
2. Evaluar el SDF en tres puntos de prueba:
   - Punto interior: $\mathbf{x}_{\text{int}} = (0.0, 0.0, 0.0)$ m $\implies$ Distancia esperada: $d \approx -0.100$ m.
   - Punto en la pared: $\mathbf{x}_{\text{wall}} = (0.100, 0.0, 0.0)$ m $\implies$ Distancia esperada: $d \approx 0.000$ m.
   - Punto exterior: $\mathbf{x}_{\text{ext}} = (0.150, 0.0, 0.0)$ m $\implies$ Distancia esperada: $d \approx +0.050$ m.
3. Calcular el gradiente numérico centrado ∇f_θ en $\mathbf{x}_{\text{wall}}$ con paso $h = 10^{-4}$ m.
4. **Criterio de Aprobación**: Las distancias evaluadas deben tener un error < 3 mm y la norma del gradiente debe satisfacer $|\|\nabla f_\theta\| - 1.0| \leq 0.04$, demostrando convergencia eikonal estricta.

7 Sprint IA-4: Extractor de Mallas Poligonales Marching Cubes C++20 con Normales Analíticas Exactas

7.1 1. Qué Estudiar en la Literatura Técnica

- **Lorensen & Cline (1987, *Marching Cubes: A High Resolution 3D Surface Construction Algorithm*)**:
  - Indexación de los 8 vértices de cada celda cúbica (patrón binario de 8 bits $\implies$ 256 configuraciones posibles).
  - Tablas de aristas intersectadas (`edgeTable`) y de conectividad de triángulos (`triTable`).
- ***Mathematik* (Tilo Arens et al.)**:
  - **Kapitel 24 & 25 (*Flächen im Raum und Gradienten*)**: Interpolación lineal a lo largo de las aristas del cubo para ubicar el conjunto de nivel cero exacto $f_\theta(\mathbf{x}) = 0$:

$$\mathbf{P}_{\text{cruce}} = \mathbf{P}_1 + \frac{0 - d_1}{d_2 - d_1}(\mathbf{P}_2 - \mathbf{P}_1) \tag{9}$$

    donde $d_1 = f_\theta(\mathbf{P}_1)$ y $d_2 = f_\theta(\mathbf{P}_2)$.

– Cálculo del vector normal unitario exacto evaluando el gradiente analítico del campo:

$$\mathbf{n}(\mathbf{P}) = \frac{\nabla f_\theta(\mathbf{P})}{\|\nabla f_\theta(\mathbf{P})\|_2} \tag{10}$$

- ***C++ professionell anwenden* (Ulrich Breymann)**:
  - **Kapitel 8 (*Sequenzielle Datenstrukturen*)**: Diseño de `MeshData3D` con búferes contiguos alineados para vértices, normales y coordenadas de textura compatibles con GPUs modernas.

7.2 2. Por Qué Hay que Implementarlo

Una representación implícita Neural SDF es una función matemática pura en memoria; para exportarla a estándares de ingeniería (glTF/GLB, AutoCAD DXF, Three.js) es indispensable convertirla a una **malla de polígonos triangulares**:

1. **Superación del Ruido de Normales de Malla Discreta:** La mayoría de los extractores de mallas calculan los vectores normales promediando el producto vectorial $(\mathbf{v}_2-\mathbf{v}_1)\times(\mathbf{v}_3-\mathbf{v}_1)$ de las caras adyacentes. En geometrías reconstruidas a partir de señales de radar con ruido, este método amplifica la rugosidad numérica y genera artefactos de sombreado rugoso no físico.
2. **Ventaja Única de las Redes Neuronales SDF:** Dado que la red f_θ es una función continua y diferenciable, podemos evaluar el gradiente exacto ∇f_θ mediante diferencias finitas centrales con paso infinitesimal $h = 10^{-4}$ m:

$$\frac{\partial f_\theta}{\partial x} \approx \frac{f_\theta(x+h,y,z) - f_\theta(x-h,y,z)}{2h} \tag{11}$$

   Esto produce normales perfectamente suaves (*smooth normals*) y matemáticamente exactas, eliminando por completo la necesidad de filtros de suavizado laplaciano posteriores que deforman las dimensiones de las tuberías.

7.3 3. Tareas Paso a Paso de Implementación

1. **Definición de estructuras de malla y cabecera del extractor**: Crear el archivo `include/gpr3d/ai/m`

```
#pragma once
#include <vector>
#include <array>
#include <functional>
#include "gpr3d/geometry/geometry_types.hpp"

namespace gpr3d::ai {

struct BoundingBox3D {
    Vector3D min_pt;
    Vector3D max_pt;
};
```

```
struct MeshData3D {
    std::vector<Vector3D> vertices;
    std::vector<Vector3D> normals;
    std::vector<uint32_t> indices;
};

class MarchingCubesExtractor {
public:
    using SDFFunction = std::function<float(const Vector3D&)>;

    // Extrae la malla triangular evaluando el campo SDF en una cuadricula adaptativa
    static MeshData3D extract(
        const SDFFunction& sdf,
        const BoundingBox3D& bbox,
        float cell_size,
        float iso_level = 0.0f
    );

private:
    static Vector3D compute_normal(const SDFFunction& sdf, const Vector3D& p, float h =
    static Vector3D interpolate_vertex(
        float iso_level,
        const Vector3D& p1, const Vector3D& p2,
        float val1, float val2
    );
};

} // namespace gpr3d::ai
```

2. **Implementación del algoritmo con tablas estáticas de indexación**: Crear el archivo `src/ai/marching_cubes.cpp` conteniendo `edgeTable[256]` y `triTable[256][16]`, iterando sobre los vóxeles de la caja delimitadora, evaluando el signo de los 8 vértices, interpolando las aristas activas y calculando las normales con la función analítica `compute_normal`.

7.4 4. Prueba de Verificación Numérica

Implementar `test/test_marching_cubes.cpp`:

1. Definir un campo analítico de una esfera perfecta de radio $R = 0.25$ m centrada en el origen: $f(x, y, z) = \sqrt{x^2 + y^2 + z^2} - 0.25$.
2. Ejecutar `MarchingCubesExtractor::extract` en la caja $[-0.35, 0.35]^3$ m con paso de celda $\Delta = 0.01$ m (1 cm).
3. Para cada vértice $\mathbf{v}$ generado, calcular su distancia al origen $r = \|\mathbf{v}\|$ y el ángulo entre la normal generada $\mathbf{n}(\mathbf{v})$ y la normal teórica $\mathbf{v}/\|\mathbf{v}\|$.
4. **Criterio de Aprobación**: El error máximo de radio debe ser $|r - 0.25| < 0.001$ m (1 mm), y la desviación angular media de las normales debe ser inferior a $0.4°$, validando la fidelidad poligonal del extractor.

8 Sprint IA-5: Red 3 (LayerTransformer) - Segmentación y Reconstrucción de Horizontes Estratigráficos

8.1 1. Qué Estudiar en la Literatura Técnica

- **Vaswani et al. (2017, *Attention is All You Need*)**:
  - Mecanismo de auto-atención escalada (*Scaled Dot-Product Attention*) y codificación posicional senoidal para secuencias temporales y espaciales.
- ***Physik für Wissenschaftler und Ingenieure* (Tipler & Mosca)**:
  - **Kapitel 15 (*Reflexion an Grenzflächen*)**: Interfaces de transición dieléctrica entre capas de suelo (asfalto $\varepsilon_{r1} \approx 4$, base de grava $\varepsilon_{r2} \approx 9$, arcilla saturada $\varepsilon_{r3} \approx 20$).
- ***Mathematik* (Tilo Arens et al.)**:
  - **Kapitel 24 (*Parametrische Flächen*)**: Superficies de elevación geológica continuas $z = h_k(x, y)$ y regularización de suavidad mediante mínima curvatura (laplaciano de la superficie).

8.2 2. Por Qué Hay que Implementarlo

El suelo nunca es un bloque homogéneo; está formado por capas superpuestas de pavimento, base y estratos naturales:

1. **Vulnerabilidad de los Detectores Clásicos de Picos:** El algoritmo tradicional de seguimiento de líneas de reflectores se interrumpe cada vez que una tubería o una piedra cruza el horizonte, generando hipérbolas de difracción que enmascaran la interfaz continua de la capa.
2. **Auto-Atención para Continuidad Espacial de Largo Alcance:** La arquitectura Transformer aprende correlaciones globales entre trazas distantes del radargrama. Si la reflexión de una capa desaparece puntualmente debido a la sombra electromagnética de una tubería metálica, la red reconstruye de forma lógica la superficie continua de la capa a través de la zona de sombra.
3. **Restricción Física de No-Intersección (Monotonía Geológica):** Los estratos geológicos se depositan secuencialmente por gravedad. Una capa superior k jamás puede penetrar físicamente por debajo de la capa inferior $k + 1$. Para hacer cumplir esto, se introduce la penalización en la pérdida:

$$\mathcal{L}_{\text{mono}} = \sum_{k=1}^{K-1} \max\left(0,\ h_k(x, y) - h_{k+1}(x, y) + \delta_{\min}\right) \tag{12}$$

   garantizando que las capas nunca se crucen en el modelo 3D.

8.3 3. Tareas Paso a Paso de Implementación

1. **Modelo PyTorch de LayerTransformer**: Crear el archivo `scripts/ai/models/layer_transformer.p` con codificador de auto-atención bidimensional sobre perfiles B-scan y cabezal de regresión continua de cotas de profundidad $h_k(x)$.

2. **Módulo C++20 de reconstrucción estratigráfica**: Crear `include/gpr3d/ai/layer_reconstructor` y `src/ai/layer_reconstructor.cpp` que convierte los perfiles de cota predichos en mallas de superficies poligonales `MeshData3D` coloreadas por material de suelo según la litología clasificada en el Sprint 9.

8.4 4. Prueba de Verificación Numérica

Implementar `test/test_layer_reconstructor.cpp`:

1. Generar un B-scan sintético de 100 trazas con dos capas continuas a profundidades teóricas $z_1 = 0.25$ m y $z_2 = 0.65$ m, introduciendo un hueco con señal nula en las trazas 40 a 55 para simular una oclusión severa.
2. Ejecutar la inferencia con `LayerReconstructor`.
3. **Criterio de Aprobación**: En la región de oclusión (trazas 40-55), la superficie predicha debe interpolar suavemente con un error absoluto máximo de profundidad inferior a ± 15 mm, y la separación $z_2(x) - z_1(x)$ debe ser estrictamente positiva (≥ 0.35 m) en todas las 100 trazas.

9 Sprint IA-6: Integración en SignalObjectTransformer y Arquitectura Resiliente de Fallback

9.1 1. Qué Estudiar en la Literatura Técnica

- ***C++ professionell anwenden*** **(Ulrich Breymann)**:
  - **Kapitel 6 (*Klassen und Polymorphie*)**: Patrón Estrategia (*Strategy Pattern*) para selección dinámica o configurable de motores de geometría.
  - **Kapitel 14 (*Robuste Programmierung und Exception-Safety*)**: Diseño de sistemas tolerantes a fallos donde los módulos críticos nunca lanzan excepciones no controladas hacia el hilo principal de renderizado.

9.2 2. Por Qué Hay que Implementarlo

En aplicaciones críticas de ingeniería civil y seguridad de excavación (evitar perforar conducciones de gas a presión o cables eléctricos de media tensión):

1. **Peligro de Alucinaciones de Redes Neuronales:** En condiciones de campo adversas (suelos extremadamente húmedos, arcillas salinas con atenuación rápida o interferencias electromagnéticas de líneas eléctricas), las redes neuronales pueden producir predicciones de baja confianza o formas degeneradas.
2. **Arquitectura Resiliente de Fallback Automático:** El sistema debe monitorizar de forma continua dos métricas objetivas de control de calidad:

$$\text{Confianza} = 0.5 \cdot \text{Score}_{\text{DetNet}} + 0.5 \cdot (1 - \min(1.0, \text{ErrorEikonal})) \tag{13}$$

   Si la confianza cae por debajo del 50%, el pipeline **desactiva transparentemente la reconstrucción por IA** y activa el algoritmo determinista geométrico clásico (triedro de Frenet-Serret del Sprint 10), registrando una advertencia en el informe de auditoría técnica sin detener la operación de la aplicación.

9.3 3. Tareas Paso a Paso de Implementación

1. **Actualización de la interfaz en SignalObjectTransformer**: Editar `include/gpr3d/geometry/sign`

```
enum class ReconstructionMode {
    DeterministicOnly,   // Cilindros y elipsoides analiticos de Frenet-Serret
    NeuralSDFOnly,       // Malla de alta fidelidad por Neural SDF
    HybridAutoFallback   // IA preferente con fallback determinista si confianza < 0.50
};

struct ReconstructionConfig {
    ReconstructionMode mode = ReconstructionMode::HybridAutoFallback;
    float confidence_threshold = 0.50f;
    float eikonal_tolerance = 0.25f;
    float marching_cubes_resolution = 0.01f; // 1 cm
};
```

2. **Implementación del conmutador de modo y auditoría**: Actualizar `src/geometry/signal_object_` para ejecutar la lógica de decisión, etiquetando cada objeto 3D con un campo booleano `is_neural_reconstruction` y guardando la puntuación de calidad numérica.

9.4 4. Prueba de Verificación Numérica

Implementar `test/test_hybrid_transformer_fallback.cpp`:

1. Inyectar un radargrama deliberadamente corrupto con ruido blanco gaussiano extremo (SNR = -10 dB) donde la red neuronal genera una confianza estimada de apenas el 22%.
2. Ejecutar `SignalObjectTransformer::transform` en modo `HybridAutoFallback`.
3. **Criterio de Aprobación**: El sistema debe completar la ejecución sin excepciones ni cuelgues, devolver un objeto de tubería con `is_neural_reconstruction = false`, y verificar que la geometría del conducto coincide con el cilindro paramétrico determinista de respaldo.

10 Sprint IA-7: Generador de Cortes Horizontales C-Scan (Depth-Slices) y Visualización Interactiva Three.js

10.1 1. Qué Estudiar en la Literatura Técnica

- ***Mathematik* (Tilo Arens et al.)**:
  - **Kapitel 24 (*Rasterung und Interpolation*)**: Interpolación bilineal sobre planos de corte $z = z_0$ a partir de una rejilla tridimensional de radar $V(x, y, z)$.
- **Three.js API y Shader Materials**:
  - Creación de planos de textura semitransparentes (`MeshBasicMaterial` con canal alfa), mapas de calor de gradiente de color falso (Paleta Viridis / Jet geofísica) y enlaces de control interactivo mediante deslizadores en la interfaz HTML.

10.2 2. Por Qué Hay que Implementarlo

1. **Validación Cruzada por el Ingeniero Geotécnico:** En la práctica pericial y forense, ningún ingeniero civil certifica un replanteo basándose únicamente en una malla 3D opaca. Necesita inspeccionar visualmente la rebanada horizontal de energía radar (*C-Scan / Depth Slice*) a la profundidad exacta del objeto para comprobar que la tubería 3D reconstruida se alinea milimétricamente con la huella de energía del radargrama.
2. **Navegación Intuitiva en el Subsuelo:** Disponer de un deslizador en el navegador web que permita cortar el terreno de 0 a 3 metros de profundidad en tiempo real, observando cómo las mallas 3D emergen y se sumergen en las láminas de corte de radar, proporciona una comprensión espacial instantánea para operadores no expertos.

10.3 3. Tareas Paso a Paso de Implementación

1. **Desarrollo del rasterizador C-Scan en C++20**: Crear `include/gpr3d/processing/cscan_rasteri` y `src/processing/cscan_rasterizer.cpp`:

```
#pragma once
#include <vector>
#include <string>
#include "gpr3d/processing/grid3d.hpp"

namespace gpr3d::processing {

struct DepthSliceImage {
    int width;
    int height;
    float depth_meters;
    std::vector<uint8_t> rgba_pixels; // Imagen lista para textura Three.js
};

class CScanRasterizer {
public:
```

```
    static DepthSliceImage generate_slice(
        const Grid3D<float>& migrated_grid,
        float target_depth,
        float velocity = 0.10f
    );
};

} // namespace gpr3d::processing
```

2. **Integración en el Dashboard Web Three.js**: Actualizar `dashboard/index.html` con un plano móvil de visualización acoplado al visor 3D:

```
// Creacion del plano de corte horizontal interactivo C-Scan
const sliceGeometry = new THREE.PlaneGeometry(surveyWidth, surveyLength);
const sliceMaterial = new THREE.MeshBasicMaterial({
    map: sliceTexture,
    transparent: true,
    opacity: 0.75,
    side: THREE.DoubleSide
});
const sliceMesh = new THREE.Mesh(sliceGeometry, sliceMaterial);
sliceMesh.rotation.x = -Math.PI / 2;
scene.add(sliceMesh);

// Evento de deslizador de profundidad (Depth Slider)
document.getElementById('depthSlider').addEventListener('input', (e) => {
    const depth = parseFloat(e.target.value);
    sliceMesh.position.y = -depth;
    document.getElementById('depthValue').innerText = depth.toFixed(2) + ' m';
});
```

10.4 4. Prueba de Verificación Numérica

Implementar `test/test_cscan_rasterizer.cpp`:

1. Construir un volumen sintético de $100 \times 100 \times 100$ con un reflector puntual de alta energía situado en la celda $(30, 70, 45)$.
2. Extraer la rebanada C-Scan a la cota correspondiente a la capa 45.
3. **Criterio de Aprobación**: El píxel con máximo valor de brillo en la imagen RGBA devuelta debe ubicarse exactamente en las coordenadas $(x = 30, y = 70) \pm 0$ píxeles, y su canal alfa debe ser 255.

11 Sprint IA-8: Conectividad Industrial DXF/IFC 4.0, Reporte de Interferencias y Validación con Radargramas Reales

11.1 1. Qué Estudiar en la Literatura Técnica

- **Norma Internacional ISO 16739-1:2018 (*Industry Foundation Classes - IFC 4.0*)**:
  - Definición de esquemas de representación geométrica mediante facetas triangulares: `IfcTriangulatedFaceSet` y asignación de clases de ingeniería civil: `IfcFlowFitting` (codos y derivaciones de tubería) e `IfcDistributionFlowElement` (conductos principales).
- **Especificación Oficial de AutoCAD DXF (Autodesk)**:
  - Cóacutedigos de grupo para la entidad `MESH` tridimensional: número de vértices (grupo 71), número de caras (grupo 72) y lista ordenada de conectividad de vértices (grupo 90).

11.2 2. Por Qué Hay que Implementarlo

1. **Entrega de Valor en el Flujo BIM Real:** Los ayuntamientos y empresas constructoras no pueden integrar capturas de pantalla o visores experimentales en sus sistemas de información geográfica corporativos (GIS/BIM). Exigen archivos estándar `.ifc` y `.dxf` que se abran sin conversión en Autodesk Revit, Bentley OpenRoads o QGIS.
2. **Detección Automática de Interferencias Críticas (Clash Detection):** Antes de iniciar una excavación con retroexcavadora en una calle urbana, la legislación exige un certificado de distancias mínimas de seguridad entre conductos de gas y canalizaciones de fibra óptica ($d \geq 0.30$ m). El software debe calcular automáticamente las distancias euclidianas mínimas entre todas las mallas generadas por la IA y alertar de posibles colisiones.
3. **Cierre del Ciclo de Vida del Software sobre Datos Reales de Campo:** Todo el pipeline (desde la lectura cruda de `parking.nx15.dzt` y `ds010_A.DZT`, pasando por filtros DSP, migración, detección 3D por DetNet y Neural SDF) debe validarse sobre radargramas de campo reales para asegurar su robustez ante el ruido del mundo real.

11.3 3. Tareas Paso a Paso de Implementación

1. **Exportador nativo IFC 4.0 en C++20**: Crear `include/gpr3d/export/ifc_exporter.hpp` y `src/export/ifc_exporter.cpp` para serializar mallas `MeshData3D` en formato STEP (ISO 10303-21) con cabecera estándar IFC 4.0.
2. **Motor de detección de interferencias y cálculo de distancias mínimas**: Crear `include/gpr3d/analysis/interference_detector.hpp` y `src/analysis/interference_detector.cp`

```
#pragma once
#include <vector>
#include "gpr3d/geometry/geometry_types.hpp"
#include "gpr3d/ai/marching_cubes.hpp"

namespace gpr3d::analysis {
```

```
struct ClashEvent {
    std::string object_id_a;
    std::string object_id_b;
    float minimum_distance_meters;
    Vector3D closest_point_a;
    Vector3D closest_point_b;
    bool is_critical; // True si distancia < margen de seguridad (ej. 0.30 m)
};

class InterferenceDetector {
public:
    static std::vector<ClashEvent> check_clashes(
        const std::vector<ai::MeshData3D>& meshes,
        const std::vector<std::string>& object_ids,
        float safety_margin = 0.30f
    );
};

} // namespace gpr3d::analysis
```

3. **Script de validación integral sobre datos reales**: Crear `scripts/ai/validate_real_dzt.py` para cargar `parking.nx15.dzt`, procesar la traza con el pipeline híbrido y exportar el informe técnico completo de interferencias y mallas IFC.

11.4 4. Prueba de Verificación Numérica

Implementar `test/test_real_pipeline_ai.cpp`:

1. Cargar el archivo de radargrama real del repositorio: `parking.nx15.dzt`.
2. Ejecutar el pipeline de extremo a extremo:
   - Filtro IIR Bandpass y corrección Time-Zero.
   - Migración de Kirchhoff 3D sobre las trazas.
   - Detección de ROI con `GPRDetNet` y reconstrucción morfológica con `GPRDeepSDF`.
   - Extracción de malla por `MarchingCubesExtractor`.
   - Generación de archivos de salida `output/parking_mesh.ifc` y `output/parking_mesh.dxf`.
3. **Criterio de Aprobación**:
   - Se deben generar los archivos IFC y DXF sin ningún fallo de segmentación.
   - El archivo IFC generado debe comenzar con la cabecera válida `ISO-10303-21;` y contener al menos una entidad `IFCTRIANGULATEDFACESET`.
   - La malla resultante debe contener más de 500 triángulos con área total no nula y sin vértices con valores `NaN` o infinitos.

12 Plan de Acción, Cronograma y Matriz de Entregables

Para asegurar un avance metódico e impecable, la implementación de las redes neuronales 3D se organiza en la matriz de fases mostrada en el Cuadro 1.

Fase / Sprint	Objetivo Principal y Entregables de Código	Archivos Clave C++ / Python	Criterio de Éxito
Sprint IA-0	Motor de inferencia autónomo en C++20 con ONNX Runtime sin dependencias de Python	`inference_engine.hpp` `inference_engine.cpp`	Latencia < 0.5 ms, Cero copias (`std::span`)
Sprint IA-1	Generador sintético procedural CAD y simulaciones FDTD batch con gprMax	`procedural_cad.py` `gprmax_batch.py`	15,000 pares HDF5, Error de pico < 0.15 ns
Sprint IA-2	Red 1 (GPR-DetNet) para detección y delimitación de cajas envolventes 3D	`gpr_detnet.py` `detnet_detector.cpp`	IoU 3D ≥ 0.78, Confianza $> 85\%$
Sprint IA-3	Red 2 (GPR-DeepSDF) con decodificador continuo y regularización Eikonal PINN	`deep_sdf_decoder.py` `sdf_evaluator.cpp`	$\|\nabla f_\theta\| \approx 1 \pm 0.04$, Error dist. $<$ 3 mm
Sprint IA-4	Extractor de mallas Marching Cubes C++20 con normales analíticas exactas	`marching_cubes.hpp` `marching_cubes.cpp`	Error radio $<$ 1 mm, Desv. normal $< 0.4°$
Sprint IA-5	Red 3 (LayerTransformer) para reconstrucción de horizontes estratigráficos	`layer_transformer.py` `layer_reconstructor.cpp`	Capas sin cruces ($h_1 < h_2 < h_3$), MAE < 15 mm
Sprint IA-6	Conmutador resiliente de fallback en `SignalObjectTransformer`	`signal_object_transformer.cpp`	Fallback transparente si Confianza $< 50\%$
Sprint IA-7	Rebanadas horizontales C-Scan y visualizador interactivo con slider en Three.js	`cscan_rasterizer.cpp` `dashboard/index.html`	Precisión de píxel ± 0, opacidad continua
Sprint IA-8	Exportación industrial IFC 4.0/DXF, reporte de colisiones y validación real	`ifc_exporter.cpp` `interference_detector.cpp`	Pipeline validado en `parking.nx15.dzt`

Table 1: Cronograma de implementación del subsistema de Inteligencia Artificial en `gpr3d`.

— FIN DE LA GUÍA TÉCNICA Y PLAN DE IMPLEMENTACIÓN IA 3D —